LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

Modul 3 Konsep Dasar Pemrograman Berorientasi Objek



RAYMUNDUS ARIEL ABAS

103132430021

S1SD05

PROGRAM STUDI SAINS DATA

FAKULTAS DIREKTORAT KAMPUS PURWOKERTO

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Konsep Dasar Pemrograman Berorientasi Objek (OOP)

Sistem yang dibangun dengan metode pemrograman berorientasi objek adalah sebuah sistem yang komponennya di-enkapsulasi menjadi kelompok data dan fungsi. Komponen-komponen tersebut dapat saling berinteraksi satu sama lain serta dapat mewarisi atribut dan sifat dari komponen lainnya. Terdapat beberapa karakteristik utama dalam OOP, yaitu:

Abstraksi: Abstraksi adalah proses menemukan hal-hal yang esensial pada suatu objek dan mengabaikan hal-hal yang sifatnya insidental. Melalui abstraksi, kita dapat menentukan apa saja ciri-ciri (atribut) dan apa saja yang bisa dilakukan (metode/perilaku) oleh suatu objek tersebut.

- Enkapsulasi: Pengkapsulan adalah proses pemaketan data objek bersama dengan method-methodnya. Enkapsulasi berfungsi untuk menyembunyikan rincian implementasi (data hiding) sehingga bagian internal program dapat diubah tanpa mempengaruhi bagian lainnya. Selain itu, enkapsulasi menciptakan modularitas (modularity) yang memungkinkan objek dikelola dan dimodifikasi secara independen.
- Pewarisan (Inheritance): Proses penciptaan kelas turunan (subclass) baru dengan cara mewarisi karakteristik berupa atribut dan operasi dari kelas induk (superclass) yang sudah ada, ditambah dengan karakteristik unik dari kelas baru tersebut.
- Polymorphisms: Berasal dari bahasa Yunani yang berarti "banyak bentuk", yang memungkinkan digunakannya sebuah antarmuka yang sama untuk memerintah objek melakukan aksi yang secara prinsip sama, namun prosesnya berbeda (satu interface banyak aksi).
- Message (Komunikasi Antar Objek): Komunikasi dan interaksi antar objek dilakukan melalui pengiriman pesan (seperti pemanggilan method) yang kerap disertai parameter informasi pendukung.

2. Kelas dan Objek

- Kelas: Kelas merupakan cetak biru atau rancangan dari sebuah tipe objek, sehingga kita bisa menciptakan banyak objek dari satu buah kelas. Anggota utama dari sebuah kelas terdiri atas atribut (field identitas) dan method (perilaku berupa prosedur maupun fungsi).
- Objek: Objek adalah hasil instansiasi atau hasil ciptaan dari suatu kelas yang telah didefinisikan sebelumnya. Objek dimungkinkan juga memiliki komposisi dari objek-objek lain atau mewarisi sifat dari objek induknya.

3. Konsep Field (Atribut)

Field merupakan anggota dari kelas yang digunakan untuk menyimpan data, dan dibagi menjadi dua jenis:

- Class Field: Field yang berkaitan langsung dengan kelas itu sendiri, bukan dengan *instance* (objek) dari kelas. Field ini dideklarasikan dengan static modifier dan penggunaannya diakses melalui pemanggilan nama kelasnya.
- Instance Field: Field yang dideklarasikan tanpa static modifier. Field ini terkait dengan suatu *instance* atau objek tertentu, sehingga setiap objek yang diciptakan akan memiliki salinan datanya masing-masing.

4. Method dan Parameter Passing

Method berfungsi untuk memodularisasi program dengan cara memisahkan berbagai tugas spesifik dalam suatu kelas. Dalam memberikan argumen/parameter pada method, terdapat dua cara pelewatan nilai, yaitu:

- Pass by Value: Digunakan pada Java untuk semua jenis data. Pada tipe data primitif, nilai yang diberikan kepada method merupakan salinan dari nilai asli. Pada objek dan array, nilai yang diberikan merupakan salinan dari reference yang menunjuk ke objek atau array yang sama. Oleh karena itu, perubahan isi objek atau array melalui reference tersebut dapat terlihat pada variabel asal.
- Pass by Reference: Digunakan pada tipe data array dan objek. Karena variabel menyimpan penunjuk (alamat) memori, maka nilai yang diubah di dalam method akan memengaruhi nilai pada variabel asalnya, sebab keduanya menunjuk ke memori objek yang sama.

5. Keyword "this"

Keyword this adalah besaran referensi khusus pada Java yang selalu merujuk pada objek tempat method itu sendiri dipanggil. Salah satu kegunaannya adalah untuk menyelesaikan konflik penamaan apabila parameter memiliki nama yang sama persis dengan properti/variabel kelas. Selain itu, keyword this dapat dimanfaatkan untuk memanggil satu konstruktor dari dalam konstruktor lain yang berada di kelas yang sama.

B. Guided

1. [Abstraksi]

Blueprint:
Kucing.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package abstraksi;

/**
 *
 * @author ACER
 */
public class Kucing {

    String nama;
    String ras;

    void bersuara() {
        System.out.println(nama + ras + "meow");
    }
}
```

Main
Abstraksi.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package abstraksi;

/**
 *
 * @author ACER
```

```
 */
public class Abstraksi {
    public static void main(String[] args) {
        Kucing aren = new Kucing();

        aren.nama = "Aren Gentong";
        aren.ras = "Mujaer";
        aren.bersuara();
    }
}
```

Output :

[

```
--- exec:3.6.3:exec (default-cli) @ guided ---
Aren GentongMujaermeow
------------------------------------------------------------------------
```

]

2. [Encapsulation]

Blueprint

Rekening.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package encapsulation;

/**
 *
 * @author ACER
 */
public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah) {
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo() {
```

```
        System.out.println("Saldo Anda: " + saldo);
    }
```

Main

Encapsulation.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package encapsulation;

/**
 *
 * @author ACER
 */
public class Encapsulation {
    public static void main(String[] args) {
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
```

Output :

```
[
--- exec:3.6.3:exec (default-cli) @ guided ---
Saldo berhasil ditambahkan
Saldo Anda: 20000000
------------------------------------------------------------------------
]
```

3. [Circle]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */

/**
 *
 * @author ACER
```

```
 */
public class Circle {

    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;

    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }
}
```

Output :
Tidak terdapat output karena kode Circle hanya mendefinisikan class dan method tanpa main()

4. Method

```
        /*
        Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
        default.txt to change this license
        Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
        this template
        */

        /**
        *
        @author ACER
        */
        public class TestPass {

        // 1. Atribut
        int i, j;

        // 2. Konstruktor
        TestPass(int a, int b) {
        i = a;
```

```
        j = b;
        }

        // 3. Method Pass by Value (primitif)
        void calculate(int m, int n) {
        m = m * 10;
        n = n / 2;
        }

        // 4. Method Pass by Reference (objek)
        void calculate(TestPass e) {
        e.i = e.i * 10;
        e.j = e.j / 2;
        }

        // 5. Entry point untuk menjalankan program
        public static void main(String[] args) {
        int x, y;
        TestPass z;
        z = new TestPass(50, 100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        // passed by value
        z.calculate(x, y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        // passed by reference
        z.calculate(z);
        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
        }
}
```

Output :

```
[
--- exec:3.6.3:exec (default-cli) @ guided ---
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
------------------------------------------------------------------------
]
```

5. DemoLagu

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */

/**
 *
 * @author ACER
 */
class Lagu {

    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {
        if (judul == null && pencipta == null) {
            return;
        }
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}

public class DemoLagu {
```

```
    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

Output:

```
[
--- exec:3.6.3:exec (default-cli) @ guided ---
Judul : God Will Make A Way, pencipta : Don Moen
------------------------------------------------------------------------
]
```

6. Buku

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */

/**
 *
 * @author ACER
 */
public class Buku {

    private String pengarang;
    private String judul;

    private Buku() {
        this("Rumah Kita", "GoodBles");
// this ini digunakan untuk memanggil konstruktor yang
// menerima dua parameter
    }

    private Buku(String judul, String pengarang) {
        this.judul = judul;
        this.pengarang = pengarang;
    }

    private void cetakKeLayar() {
        System.out.println("Judul : " + judul + " Pengarang : "
                + pengarang);
```

```
    }

    public static void main(String[] args) {
        Buku a, b;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output:

```
[
--- exec:3.6.3:exec (default-cli) @ guided ---
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
--------------------------------------------------------------------------------
]
```

C. Unguided

1. Soal Unguided — Modul 3: Pemrograman Berorientasi Objek

Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga this perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: instance field (milik masing-masing object) dan class field (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di main sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka -1.0 di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Kebutuhan program

Class ini harus dapat:

1. Menyimpan data suhu 7 hari, dibuat dari array suhu mentah yang sudah ada di main (index array mewakili hari).
2. Menyimpan nilai penanda data kosong (-1.0) sebagai aturan tetap milik class, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. Menampilkan seluruh data apa adanya, menandai hari yang kosong secara jelas.
4. Mencari apakah ada hari yang kosong, dan di index berapa, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. Mengisi hari yang kosong, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. Menghitung rata-rata suhu dari data yang sudah bersih.

Hint

Hint di bawah ini adalah acuan minimal. Anda tetap boleh menambah method atau menyesuaikan detail selama seluruh kebutuhan di atas terpenuhi.

Nama class: PengolahSuhu

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
suhuHarian	Instance field	double[]	Menyimpan array suhu 7 hari; berbeda untuk setiap object
NILAI_KOSONG	Class field	double	Nilai penanda data kosong (-1.0); sama untuk semua object dan tidak berubah

Method yang disarankan:

Method	Kebutuhan	Return
tampilkanData()	3	void
cariIndexKosong()	4	int — index hari yang bernilai NILAI_KOSONG, atau -1 apabila tidak ditemukan
isiDataKosong()	5	void — memanggil cariIndexKosong() di dalamnya; jika hasilnya bukan -1, isi index tersebut
hitungRataRata()	6	double

Rumus imputasi (pengisian data kosong) untuk time series:

Untuk index hari i yang kosong:

suhuHarian[i] = (suhuHarian[i-1] + suhuHarian[i+1]) / 2

Ini adalah metode imputasi paling dasar untuk data deret waktu, yaitu mengisi nilai yang hilang dengan rata-rata dari titik data tetangga terdekatnya (sebelum dan sesudah), dengan asumsi nilai yang hilang berada di antara dua kondisi yang sudah diketahui dan tidak berubah drastis.

Data suhu (array di main)

Gunakan array berikut sebagai data suhu 7 hari (Hari 1 = index 0):

double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

Hari yang kosong: Hari 4.

Alur program di main

1. Siapkan array suhuHarian seperti di atas.
2. Buat object PengolahSuhu dari array suhuHarian tersebut. Tampilkan data awal (tampilkanData()), lalu panggil cariIndexKosong() dan tampilkan index yang ditemukan.
3. Jalankan isiDataKosong().
4. Tampilkan data akhir yang sudah bersih (tampilkanData()) beserta rata-rata suhunya.
5. Tampilkan kembali isi array suhuHarian di main (bukan lewat object PengolahSuhu, melainkan variabel array yang sudah disiapkan di langkah 1). Amati bahwa array ini pun sudah berubah, padahal tidak pernah diubah langsung di main, lalu jelaskan sebabnya dalam beberapa kalimat sebagai komentar pada kode program.

Output yang diharapkan

=== Data Suhu Awal ===

Hari 1 : 30.4°C

Hari 2 : 24.3°C

Hari 3 : 26.8°C

Hari 4 : (kosong)

Hari 5 : 31.4°C

Hari 6 : 30.8°C

Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===

Hari 1 : 30.4°C

Hari 2 : 24.3°C

Hari 3 : 26.8°C

Hari 4 : 29.1°C

Hari 5 : 31.4°C

Hari 6 : 30.8°C

Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:

[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]

(ikut berubah: constructor menyimpan referensi array yang sama)

Yang perlu diperhatikan saat mengerjakan

- Amati baik-baik dua baris terakhir pada output di atas. Array suhuHarian yang dicetak itu adalah variabel di main, bukan diambil lewat object PengolahSuhu — dan nilainya sudah berubah, padahal main tidak pernah memanggil operasi pengubahan itu sendiri. Ini terjadi karena array bertipe reference: constructor PengolahSuhu menyimpan array yang sama persis (bukan menyalinnya), sehingga perubahan yang dilakukan isiDataKosong() pada object otomatis terlihat juga dari sisi main. Jelaskan hal ini dalam beberapa kalimat sebagai komentar pada kode program.

- Nilai penanda -1.0 harus ditulis satu kali saja sebagai class field, lalu dipakai oleh setiap method yang membutuhkannya. Jangan mengetik ulang angka -1.0 di dalam method.
- Rumus imputasi mengasumsikan hari sebelum dan sesudah hari yang kosong tersedia sebagai data asli (bukan ikut kosong). Data pada soal ini sudah dijamin memenuhi asumsi tersebut.

Program:

Pengolahsuhu.java:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package raymundusariel;

/**
 *
 * @author ACER
 */
public class PengolahSuhu {

    // Instance field
    private double[] suhuHarian;

    // Class field: nilainya tetap dan digunakan bersama semua object
    public static final double NILAI_KOSONG = -1.0;

    // Constructor
    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    // Menampilkan data suhu
    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {

            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println(
                    "Hari " + (i + 1) + " : " + suhuHarian[i] + "\u00B0C"
```

```
                );
            }
        }
    }

    // Mencari index data yang kosong
    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {

            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }

        return -1;
    }

    // Mengisi data yang kosong menggunakan rata-rata
    // suhu hari sebelum dan sesudahnya
    public void isiDataKosong() {

        int indexKosong = cariIndexKosong();

        if (indexKosong != -1) {
            suhuHarian[indexKosong] =
                (suhuHarian[indexKosong - 1]
                + suhuHarian[indexKosong + 1]) / 2;
        }
    }

    // Menghitung rata-rata suhu
    public double hitungRataRata() {

        double total = 0;

        for (double suhu : suhuHarian) {
            total += suhu;
        }

        return total / suhuHarian.length;
    }
}
```

Main.java

```
/*
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package raymundusariel;

/**
 *
 * @author ACER
 */
import java.util.Arrays;

public class Main {

    public static void main(String[] args) {

        // Data suhu mentah selama 7 hari
        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };

        // Membuat object PengolahSuhu
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        // Menampilkan data awal
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        // Mencari index data yang kosong
        int indexKosong = pengolah.cariIndexKosong();

        System.out.println();
        System.out.println(
                "Index hari kosong (dimulai dari 0): " + indexKosong
        );

        // Mengisi data yang kosong
        pengolah.isiDataKosong();

        // Menampilkan data setelah pengisian
        System.out.println();
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();

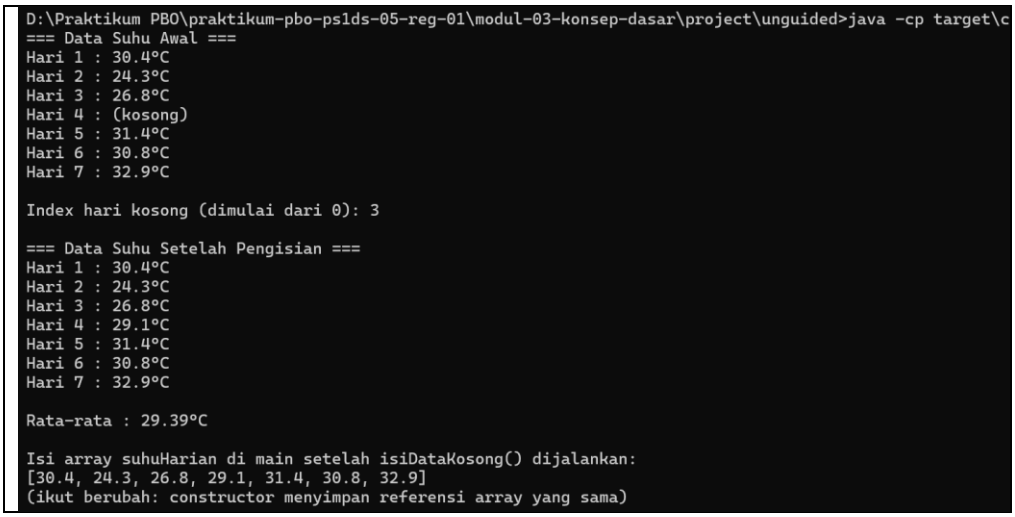
        // Menghitung dan menampilkan rata-rata
```

```
        double rataRata = pengolah.hitungRataRata();

        System.out.printf(java.util.Locale.US, "%nRata-rata :
%.2f\u00B0C%n", rataRata);

        // Menampilkan kembali array yang ada di main
        System.out.println();
        System.out.println(
                "Isi array suhuHarian di main setelah isiDataKosong()
dijalankan:"
        );
        System.out.println(Arrays.toString(suhuHarian));
        System.out.println("(ikut berubah: constructor menyimpan referensi
array yang sama)");
        /*
         * Array suhuHarian di main ikut berubah karena array merupakan
         * tipe data reference.
         *
         * Saat object PengolahSuhu dibuat, constructor tidak membuat
         * array baru, tetapi menyimpan referensi ke array suhuHarian
         * yang sama di main melalui:
         *
         * this.suhuHarian = suhuHarian;
         *
         * Oleh karena itu, ketika isiDataKosong() mengubah isi array
         * melalui object PengolahSuhu, perubahan tersebut juga terlihat
         * pada variabel suhuHarian yang berada di main.
         */
    }
}
```

Output:



Penjelasan:

Tugas ini membuat class PengolahSuhu untuk mengolah data suhu selama 7 hari menggunakan konsep dasar Pemrograman Berorientasi Objek (PBO).

Data suhu disimpan dalam array suhuHarian, dengan nilai -1.0 sebagai penanda bahwa data pada hari tersebut kosong. Program kemudian:

1. Menampilkan data suhu awal dan menandai hari yang datanya kosong.
2. Mencari index data kosong menggunakan method cariIndexKosong(). Pada data yang diberikan, data kosong berada pada index 3 atau Hari 4.
3. Mengisi data yang kosong menggunakan rata-rata suhu hari sebelumnya dan sesudahnya:
   (26.8 + 31.4) / 2 = 29.1°C
4. Menghitung rata-rata suhu setelah data kosong diisi, yaitu 29.39°C.
5. Menampilkan kembali array pada main untuk menunjukkan bahwa array ikut berubah, karena constructor menyimpan referensi ke array yang sama menggunakan this.suhuHarian = suhuHarian.

   Dengan demikian, tugas ini menerapkan class, object, constructor, instance field, class field (static final), method, array, this, serta konsep reference pada array.

D. Kesimpulan

Berdasarkan praktikum Modul 3 tentang Pemrograman Berorientasi Objek, dapat disimpulkan bahwa konsep dasar OOP dapat diterapkan melalui penggunaan kelas, objek, field, method, constructor, dan keyword `this`. Pada praktikum ini, class `PengolahSuhu`

digunakan untuk mengolah data suhu harian dengan memisahkan data dan fungsi pengolahannya ke dalam sebuah objek.

Praktikum juga menunjukkan penerapan instance field melalui `suhuHarian` dan class field melalui konstanta `NILAI_KOSONG`. Data suhu yang kosong dapat dicari berdasarkan index, kemudian diisi menggunakan nilai rata-rata dari data sebelum dan sesudahnya. Selain itu, penggunaan `this.suhuHarian = suhuHarian` menunjukkan bahwa objek menyimpan referensi array yang sama dengan array pada `main`, sehingga perubahan data di dalam objek juga terlihat pada array asal.

Dengan demikian, praktikum ini memberikan pemahaman mengenai penerapan konsep dasar Pemrograman Berorientasi Objek dalam membangun program yang lebih terstruktur, modular, dan mudah dikelola.